LLM Model Update as Mutation Event: Formalizing What Constitutes a Mutation Event in CKS Architecture

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 12, 2026

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in "The Instinct/Reasoning Separation Outside the Model" (Li, April 2026), the second paper in the CKS theory series following "Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems" (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the precise architectural meaning of the term mutation event as that term is used in the source papers, so that downstream mutation governance machinery has a formally defined activation trigger.

Series position: Derivation note B2.61. Phase B2 operational variant, opening the B1.13 mutation decomposition. Sixty-first note in Phase B2. Prior notes B2.56–B2.60 completed the B1.12 three-mechanisms-integrating-frame decomposition. B2.61 opens a six-note decomposition of B1.13 covering: mutation event definition (this note), mutation detection operational specification (B2.62), verification gate triggering per B2.06 (B2.63), routing adaptation per B2.04 (B2.64), pinning enforcement per B2.05 (B2.65), and mutation governance verification (B2.66). Phase B2 continues after B2.66 with B1.14 directed selection decomposition (B2.67–B2.72 and beyond).

Abstract

The CKS architecture recognizes three evolution mechanisms operating in productive tension: instinct evolution (mutation), DNA evolution (directed selection), and action-feedback evolution. Mutation governance — verification, routing, and pinning — applies to the first mechanism. That governance framework requires a formally defined activation trigger: a precise statement of what constitutes a mutation event. This note provides that definition. In CKS architecture, a *mutation event* is any change to the LLM model version, weights, or capabilities that cells consult. Five types of mutation event are recognized: vendor-initiated model update, vendor model improvement within a version label, LLM vendor migration per A1.05, forced migration per A6.03, and fine-tuning of the LLM's weights. Model deprecation constitutes a mutation event at the point it drives migration. The operational test is singular: does this change the instinct layer that cells consult? If yes, it is a mutation event, and mutation governance applies. All mutation events originate outside CKS substrate — in the LLM vendor's training and deployment pipeline — which is what makes boundary governance the appropriate governance shape. The note distinguishes mutation events (classification) from mutation effects

(outcomes), articulates the biological analog of undirected genetic mutation as conceptual scaffold, and identifies the precise limit at which fine-tuning is mutation rather than directed selection.

1. Why formalizing the mutation event definition is necessary

The CKS architecture commits to three evolution mechanisms in productive tension (Paper 2, §7.2). One mechanism — instinct evolution — operates through what the source paper names as mutation-like change: capability upgrades that arrive from upstream and are undirected from the deployment's perspective. Paper 2's §8 specifies that this mechanism is governed through verification, routing, and orchestration rules that pin high-stakes decisions to the reasoning layer. That governance commitment names the governance shape. It does not, by itself, specify the activation trigger — the precise architectural definition of what event activates the governance framework.

Without a defined activation trigger, mutation governance cannot operate. A deployment that lacks a formal definition of mutation event either applies governance inconsistently — applying it to some LLM changes but not others, with no principled boundary — or fails to apply it at all. The governance machinery described in subsequent notes (B2.62 through B2.66) requires this note's definition as its precondition.

This note is the first of six notes decomposing B1.13. Its specific contribution is to formalize the mutation event definition as the prerequisite on which the detection, verification, routing, pinning, and governance-verification notes depend. The strategic purpose of formalizing this as a standalone derivation is to establish the definition as independent prior art, so that the full scope of what counts as a mutation event — including vendor migration, forced migration, fine-tuning, and model deprecation — is publicly documented under the author's name.

2. The architectural definition of mutation event

In CKS architecture, a **mutation event** is any change to the LLM model version, weights, or capabilities that cells consult.

The definition has three components. *LLM model version* refers to the version identifier the deployment uses to access the instinct layer — for example, a named model release from a commercial LLM provider. *Weights* refers to the actual parameters of the model; a change to weights constitutes a mutation event even when the version label remains unchanged. *Capabilities* refers to the observable behavior space of the model — the range of tasks it performs, the patterns it applies, the outputs it produces. A deployment consults the instinct layer when its cells route queries, tasks, or reasoning steps to the LLM. A change is a mutation event if cells are exposed to a different LLM than the one they were exposed to before.

The operational test is singular: **does this change the instinct layer that cells consult?** If yes, it is a mutation event. If no, it is not. The test is binary and applies uniformly across all change types. The test's simplicity is intentional — it must be applicable at the point of detection (B2.62) without

requiring prior knowledge of the change's downstream effects.

2.1 Five mutation event types

Type 1: Vendor-initiated model update. The LLM vendor releases a new model version with updated weights, capabilities, or behavior. The deployment was consulting model version V1; vendor makes V2 available. This constitutes a mutation event when V2 is considered for integration. It does not constitute a mutation event while V1 remains the consulted model — the event is triggered at integration decision, not at vendor announcement.

Type 2: Vendor model improvement within a version label. The LLM vendor improves an existing model version — same version name, updated weights or behavior — without releasing a new version identifier. Deployments that consume a vendor's model under a floating version reference (always latest within a named version) may experience capability changes without explicit version migration. Because the instinct layer has changed — different weights, potentially different behavior — this constitutes a mutation event even when the version label has not changed. This type is architecturally important because it is the mutation event most likely to arrive unannounced.

Type 3: LLM vendor migration (per A1.05). The deployment migrates from one LLM vendor's model to another vendor's model. Tool-agnosticism (A1.05) supports migration by requiring the architecture to function across LLM choices. Vendor migration changes the instinct layer composition — the instinct layer is now a different model from a different vendor — and therefore constitutes a mutation event. Mutation governance applies at migration, not after it. The migration decision is governed; the vendor's model is consumed under verification.

Type 4: Forced migration (per A6.03). When a vendor becomes unavailable — through service discontinuation, contract termination, or other vendor-side events — the deployment must migrate to a replacement instinct layer. Forced migration is a mutation event with a compressed governance timeline. The mutation governance framework applies; what changes is the urgency and available timeline for verification. Governance intensity is a deployment configuration; mutation event classification is not. A forced migration that bypasses mutation governance is an architecture violation, not a governance option.

Type 5: Fine-tuning of LLM weights. When the deployment's LLM is fine-tuned — trained further on deployment-specific data — the fine-tuned model is a new instinct layer version. Fine-tuning changes the model's weights and therefore its capabilities; it satisfies the mutation event test. The fine-tuned model must be integrated through mutation governance, not treated as equivalent to the pre-fine-tuning model. Fine-tuning that only modifies the DNA layer — orchestration rules, routing configuration, substrate content — without changing LLM weights is directed selection (B1.14), not mutation. The distinction is tracked by the locus of change: LLM weights changed means mutation; CKS substrate content changed means directed selection.

2.2 Model deprecation as mutation event trigger

Model deprecation is not itself a fifth mutation event type but a trigger for Type 1 or Type 3. When a vendor deprecates a model version and requires migration to a newer version, the deprecation-driven migration is a mutation event at the point of integration. The deprecation announcement creates a governance timeline; the migration constitutes the mutation event. Deployments that plan mutation governance in advance of deprecation deadlines are exercising prospective mutation governance — configuring detection and verification before the forced migration window arrives.

3. What makes LLM-model-update-as-mutation-event architecturally distinctive

Conventional AI architectures frequently treat model updates as routine maintenance. The pattern is: update the model, redeploy the system, verify in production. There is no architectural classification of the model update as a first-class evolution event. There is no formal governance framework that activates at the moment of classification. There are engineering practices — canary deployments, blue-green releases, rollback procedures — but these are deployment hygiene practices, not architectural primitives co-determined with mechanism shape under a unified authority architecture.

CKS makes LLM model updates first-class evolution events. The classification is architecturally load-bearing because it is what activates the mutation governance framework. Without defined mutation events, there is no activation trigger — the governance machinery described in B2.62 through B2.66 has nothing to operate on. The classification is therefore not merely taxonomic; it is the architectural commitment that makes mutation governance possible.

This distinction has a precise consequence. In an architecture that treats model updates as routine maintenance, the question “which model are we running?” is a configuration question answered by deployment records. In CKS architecture, the question “which model version are we consulting?” is a governance question answered by mutation event records in substrate provenance. The substrate records not just what model is in use but when the integration was decided, what verification was performed, what routing changes were made, and what pinning decisions were reviewed. This record is mutation governance in its operational form.

The architectural contribution is the classification itself, combined with the commitment that the classification activates a formal governance framework. Engineering practices that share structural shape with verification-substrate machinery — canary deployments, shadow-launch patterns — are recognizable as related practice; they are not the same as CKS mutation governance, which commits to the governance framework as an architectural primitive with specific mechanism shape rather than as a deployment-hygiene choice.

4. The biological analog as conceptual scaffold

Paper 2’s instinct evolution mechanism is explicitly described as “mutation-like” (§7.2). The biological analog functions as a conceptual scaffold for the architectural commitment.

In biological evolution, genetic mutation is undirected: the organism does not choose which mutations occur in its genome. Mutations arrive from external processes — replication errors, radiation, chemical mutagens — and the organism experiences them as changes to its genetic material, which may be beneficial, neutral, or harmful. The organism's fitness depends on how it responds to the mutations that arrive, not on its ability to prevent them.

LLM model updates in CKS architecture are structurally analogous. The deployment does not choose when the LLM vendor releases a new model version, updates weights within an existing version, or deprecates a model the deployment has been using. These events arrive from the vendor's domain — outside CKS substrate, outside the deployment's control. They may improve the deployment's behavior, maintain it, or degrade it. The deployment's architectural health depends on how mutation governance responds to the events that arrive, not on any capacity to prevent them.

The analog is precise along three axes. First, undirectedness: just as organisms do not choose their genetic mutations, deployments do not choose vendor model updates. Second, external origin: just as mutations arise from processes outside the organism's biology, LLM updates arise from processes outside CKS substrate. Third, valence uncertainty: just as mutations may be beneficial, neutral, or harmful to an organism, LLM updates may improve, maintain, or degrade deployment behavior — and which outcome obtains is not knowable without verification.

The analog functions as a conceptual scaffold; it does not carry architectural load. The architectural substance is the classification of LLM version change as a first-class evolution event requiring governed integration, and the specification of what governance shapes apply. The biological framing helps explain why CKS uses the term *mutation* rather than the more neutral term *update* — it is not aesthetic preference but conceptual precision: mutation captures the undirected, externally-sourced, valence-uncertain character of LLM model change that the governance framework must handle.

Where the analog reaches its limits: CKS exceeds biology in that mutation governance is designed by humans, not by evolutionary pressure. The deployment can configure what verification is required before a mutation event is integrated. It can configure routing to route around known-degraded instinct responses. It can configure pinning to keep high-stakes decisions in the reasoning layer regardless of instinct capability. The organism subject to genetic mutation has no equivalent architectural design authority over how mutations are handled. CKS deployments have full design authority over mutation governance; the undirectedness is in the mutation events themselves, not in the governance response.

5. Inherited Paper 1 commitments

Four Paper 1 commitments are directly load-bearing for mutation event definition and governance.

A1.04 (AI-as-substrate-mediator). The LLM functions as the instinct layer — the substrate mediator performing high-dimensional pattern-matching under substrate governance. This is the commitment that makes mutation events architecturally significant: the instinct layer is load-bearing infrastructure

for cell operation. A change to the instinct layer is therefore a change to a governed architectural component, not merely a software dependency update. The AI-as-substrate-mediator commitment is what elevates LLM model change from infrastructure maintenance to evolution event.

A1.05 (tool-agnosticism). The architecture supports LLM vendor migration by requiring that the substrate pattern hold across different LLM tool choices. Tool-agnosticism is what makes LLM vendor migration architecturally possible. Mutation governance is what makes it architecturally safe. Together, A1.05 and mutation governance permit deployments to migrate across vendors under governance rather than being locked into a single vendor's model trajectory. Vendor migration is a mutation event because the instinct layer changes; A1.05 is the commitment that makes the migration architecturally coherent.

A6.03 (vendor unavailable boundary). When a vendor becomes unavailable, forced migration is a mutation event with a compressed governance timeline. A6.03 establishes that vendor unavailability is a recognized boundary condition requiring architectural response. Mutation governance applies at the forced migration boundary; the governance timeline may be compressed but the classification as mutation event is not relaxed. The architecture acknowledges that vendor unavailability is a category of instinct-layer disruption that deployments must be prepared to handle through the mutation governance framework.

A6.09 (vendor data-handling policy change). Vendor policy changes — including changes to data handling, retention, or model training practices — may drive deployment decisions to migrate to a different model or a different vendor. When a policy change triggers a model update or vendor migration, the resulting change to the instinct layer is a mutation event. The policy change itself does not constitute a mutation event; the resulting change to the consulted LLM does.

A2.40 (provenance records). Mutation integration events are recorded as governance events in substrate provenance. The provenance record for a mutation integration event captures: which model version was integrated, when the integration decision was made, what verification results were recorded, what routing changes were made at integration, and what pinning decisions were reviewed. This record is what makes mutation governance traceable and auditable. Without provenance recording of mutation events, the substrate cannot provide the path retraceability that the CKS determinism contract (A1.10) requires for instinct layer changes.

6. Operational implications

Four operational implications follow from the mutation event definition.

Configure mutation event detection. Deployments configure mutation event detection as the operational precondition for mutation governance. Detection is the subject of B2.62. The mutation event types named in §2.1 identify what detection must monitor: version label changes, weight updates within a version label, vendor migration decisions, forced migration triggers, and fine-tuning

operations. A deployment that does not detect mutation events cannot apply mutation governance; detection configuration is therefore the first operational step in instantiating the mutation governance framework.

Different mutation event types may warrant different governance intensities. A routine vendor-initiated model update in a stable deployment context has a different governance intensity profile than a forced migration under vendor unavailability. The mutation event classification applies uniformly — all five types are mutation events. The governance intensity — verification depth, verification timeline, routing review scope — is a deployment configuration appropriate to the urgency and the operational context. Paper 2's governance shapes do not mandate a single intensity; they mandate that the governance shapes apply to all mutation events.

Compressed governance timelines for urgent cases. Forced migration per A6.03 may require a compressed governance timeline — the vendor is unavailable and the deployment must migrate. Compressed timeline does not mean governance exemption. The mutation governance framework applies; what changes is the timeline and potentially the verification scope. Deployments that have conducted advance verification of candidate replacement models — prospective mutation governance before the forced migration trigger — are better positioned to maintain governance under compressed timelines.

Consistent framework across all mutation event types. The mutation governance framework — verification per B2.06/B2.63, routing adaptation per B2.04/B2.64, pinning enforcement per B2.05/B2.65 — applies to all five mutation event types. The consistent framework is what makes the governance commitment reliable: deployments do not have discretion to classify some LLM changes as non-mutation-events to avoid governance. Classification follows from the test, not from governance convenience.

7. Limits of the mutation event definition

Five limits require precise statement.

Classification activates governance; it does not predict harm. A mutation event is any change to the instinct layer that cells consult. Classification as a mutation event does not carry an implication that the change is harmful. Vendor model improvements may significantly enhance deployment behavior. LLM vendor migrations may produce a better-performing instinct layer. Fine-tuning on deployment-specific data may sharpen the instinct layer for the deployment's specific task domain. The classification is neutral with respect to outcome; it activates governance so that the actual effect can be determined through verification rather than assumed.

Mutation event is not the same as mutation effect. The mutation event is the change to the instinct layer. The mutation effect is what the change does to cell behavior, output quality, and deployment performance. The mutation event is classifiable before integration; the mutation effect is determinable

only through verification (B2.63). Mutation governance is governance over the integration of mutation events; verification is the mechanism by which effects are assessed before integration is confirmed.

Classification is the first step; verification, routing, and pinning follow. Mutation event classification (this note) is the architectural precondition for the five subsequent notes in the B1.13 decomposition. Detection (B2.62) determines that a mutation event has occurred. Verification gate triggering (B2.63) activates the verification substrate. Routing adaptation (B2.64) adjusts cell routing in response to the new instinct layer. Pinning enforcement (B2.65) confirms that high-stakes decisions remain in the reasoning layer. Mutation governance verification (B2.66) confirms that the full governance framework was applied. Classification alone does not constitute governance; it initiates the governance sequence.

Fine-tuning that changes LLM weights is a mutation event; fine-tuning that changes only DNA-layer rules is directed selection. This distinction is architecturally significant. When a deployment configures its LLM through fine-tuning that changes the model's actual parameters, the instinct layer has changed — a mutation event has occurred. When a deployment modifies its orchestration rules, routing configuration, or substrate content — even through automated optimization informed by action feedback — without changing the LLM's weights, the change is in the CKS substrate layer. That change is directed selection (B1.14), not mutation. The governance shapes differ: mutation requires verification, routing review, and pinning confirmation; directed selection operates under the standard authority architecture. The test for classification is the locus of change, not the method of change.

Mutation event classification does not apply to DNA-layer changes. Changes to orchestration rules, substrate content, routing configuration, and cell behavior that do not involve changes to the LLM model version, weights, or capabilities are not mutation events. They are directed selection events (B1.14) or action-feedback evolution outcomes (B1.12). Mutation governance applies only to mutation events. Applying mutation governance to DNA-layer changes would conflate the three evolution mechanisms and misapply the governance shapes Paper 2 specifies for each mechanism.

8. The operational test

A change to the CKS architecture constitutes a **mutation event** if and only if it changes the LLM model version, weights, or capabilities that cells consult — that is, if it changes the instinct layer per B2.01.

The test applies to: vendor-initiated model version updates (Type 1), vendor weight or behavior updates within a version label (Type 2), LLM vendor migrations per A1.05 (Type 3), forced migrations per A6.03 (Type 4), and fine-tuning operations that change LLM weights (Type 5). The test does not apply to changes that affect only CKS substrate content, orchestration rules, or DNA-layer configuration.

When the test is satisfied, mutation governance applies: detection configuration (B2.62), verification gate triggering (B2.63), routing adaptation (B2.64), pinning enforcement (B2.65), and governance

verification (B2.66).

9. Why naming the mutation event definition as standalone matters

The mutation event definition is formalizable as a standalone derivation because the definition is prior to and independent of the governance machinery that operates on it. A deployment could, in principle, recognize the definition without having fully specified the detection, verification, routing, and pinning mechanisms that constitute mutation governance. Stating the definition separately preserves the intellectual priority of the classification commitment — the recognition that LLM model updates are first-class evolution events requiring formal governance — independently of any specific implementation of the governance machinery.

The strategic prior-art posture of this note is to establish the full scope of what counts as a mutation event as public prior art under the author's name. The five mutation event types named in §2.1 — vendor-initiated update, within-version improvement, vendor migration, forced migration, and fine-tuning — represent the complete taxonomy of instinct-layer changes that CKS architecture recognizes as mutation events. Any subsequent claim to novel invention in a governance framework for LLM model updates must contend with this taxonomy as prior art.

B2.61 opens the six-note B1.13 decomposition. The decomposition proceeds from definition (this note) through operational specification: B2.62 addresses how deployments configure mutation event detection; B2.63 addresses how detected mutation events trigger the verification gate per B2.06; B2.64 addresses routing adaptation per B2.04 at the moment of mutation integration; B2.65 addresses pinning enforcement per B2.05 to confirm that high-stakes decisions remain in the reasoning layer; B2.66 addresses the full mutation governance verification workflow confirming that the governance framework was applied. The six notes together constitute the full operational decomposition of Paper 2's mutation governance commitment.

After B2.66, Phase B2 continues with B1.14 directed selection decomposition beginning at B2.67. The directed selection decomposition will cover the DNA evolution mechanism — the counterpart mechanism to mutation in Paper 2's three-mechanism productive tension framework. Where mutation is undirected and arrives from outside the substrate, directed selection is explicit, goal-directed, and operates on CKS substrate content under the standard authority architecture from Paper 1. The governance shapes differ; the architectural commitment to both mechanisms is what produces the productive tension Paper 2 defends.

Source paper citation

Li, Wenxin. "The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance." April 2026. §7.2 (three evolution mechanisms, instinct evolution as mutation-like), §7.3 (multi-level simultaneous evolution),

§8 (governance shapes across evolution mechanisms, specifically mutation governance through verification, routing, and pinning), §8.2 (verification substrates as governance over instinct integration), §8.3 (instinct/reasoning boundary as governed substrate content).

Li, Wenxin. “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.” April 2026. §4.1–4.2 (AI-as-substrate-mediator, A1.04), §7.1 (tool-agnosticism, A1.05), §3.1 (provenance metadata, A2.40), §6 (boundary cases including vendor unavailable A6.03 and vendor data-handling policy change A6.09).

Self-citation (series position)

This note is B2.61 in the CKS derivation note series. It follows B2.56–B2.60 (B1.12 three-mechanisms-integrating-frame decomposition complete) and opens the B1.13 mutation decomposition. Preceding foundational notes: B1.13 (multi-level simultaneous evolution), B1.10 (instinct evolution as undirected mutation), B2.01 (instinct layer operational definition), B2.04 (routing around bad instinct), B2.05 (conflict preservation and silent-merge prevention), B2.06 (corrective signal without weight modification), B2.56 (three mechanisms integrating frame). Cross-series inheritance: A1.04 (AI-as-substrate-mediator), A1.05 (tool-agnosticism), A6.03 (vendor unavailable), A6.09 (vendor data-handling policy change), A2.40 (provenance records). Subsequent notes: B2.62 (mutation detection operational specification), B2.63 (verification gate triggering), B2.64 (routing adaptation), B2.65 (pinning enforcement), B2.66 (mutation governance verification).